Does the planning tool survive reasoning mode? — think=on

5 language models · 5 PDDL planning tasks · with vs. without a real planner/validator tool · success rates with 95% confidence intervals · think=on companion to the think=off headline deck

The headline result — the baseline reasons itself to death

Reasoning mode shares one 8,192-token decode budget with the answer — and the unaided model spends it thinking instead of answering:

55–83%

of no-tools think=on trials hit the 8,192-token cap (≥9B, by task)

80% → 21%

validate_plan, Qwen3.5-9B — no-tools success, think=off → think=on

93%

the same think=on cell with the tool + nudge

The exception is solve, where reasoning genuinely helps the unaided baseline (9B 11→27%, 35B 9→38% vs think=off). Everywhere else the tool arms barely notice reasoning mode — the tool call replaces the long derivation. Every think=on number is budget-confounded; the think=off deck is the clean read.

Scorecard — six research questions, six verdicts

RQ	question	verdict	evidence (≥9B, think=on)
RQ0.1	does the tool help validate domain/problem files?	**YES**	availability +21…+74pp; 3/3 sig-for, 0/3 sig-against
RQ0.2	does the tool help SOLVE?	**YES**	availability +19…+40pp; 3/3 sig-for, 0/3 sig-against
RQ0.3	does the tool help check a plan?	**MIXED**	availability -10…+44pp; 2/3 sig-for, 1/3 sig-against
RQ0.4	does the tool help track state (simulate)?	**YES**	availability +44…+96pp; 3/3 sig-for, 0/3 sig-against
RQ0.5	does the advantage grow with plan length?	**YES**	solve gap widens +48 → +57 → +65pp
RQ0.6	does the advantage track object count?	**NO**	validate_problem gap flat (+60 / +56 / +55pp)

What we're testing

PDDL is the standard formal language AI planners use to describe a world, the actions allowed in it, a goal to reach, and step-by-step plans. We give a language model five PDDL jobs:

- solve — find a plan (a sequence of actions) that reaches the goal
- validate_domain — check that the file defining the world and its actions is correct PDDL
- validate_problem — check that the file listing the objects, the start state and the goal is correct PDDL
- validate_plan — decide whether a given plan actually works (a yes/no verdict)
- simulate — track how the world changes as a plan is run, one action at a time

The question: do models do these jobs better when we also let them call a real planner/validator program — a “tool” — instead of answering only from their own knowledge?

The three setups we compare

Every job is run in three setups (we call them “arms”), and we never mix their results:

- no-tools — the model answers on its own, with no external help. This is the baseline.
- tool available — exactly the same request, but now a real planner/validator is there for the model to call if it chooses to.
- tool + nudge — the tool is available AND we append one sentence explicitly telling the model to use it (we call this “steered”).

Comparing neighbouring arms answers two separate questions:

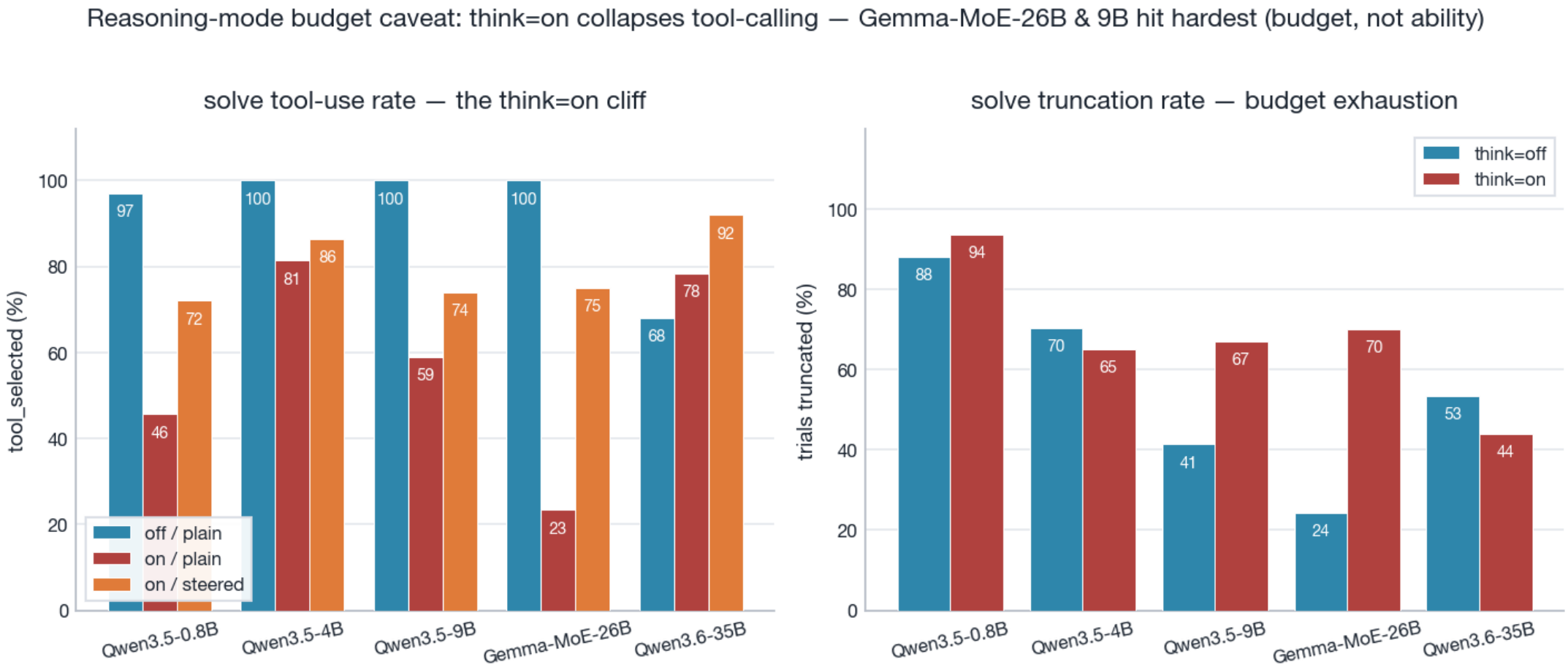
- Does simply having the tool help? (no-tools vs tool available — the wording is otherwise identical)
- Or did the model just need to be told to use it? (tool available vs tool + nudge)

In the tables these arms appear as the short codes nt-neut · tl-neut · tl-ster (same order).

How to read the results

- success rate — how often the model's answer matched the known-correct answer, 0–100%. It is the only score shown; arms are scored separately and never pooled.
- Error bars (charts) and [low, high] brackets (tables) are Wilson 95% confidence intervals. When two ranges don't overlap, the difference is real rather than noise; a "*" on a gap marks exactly that. Green = the tool helped, red = the tool hurt.
- "≥9B" — headline conclusions use the larger models (Qwen3.5-9B, Gemma-MoE-26B, Qwen3.6-35B); Qwen3.5-4B is shown for context and the 0.8B failure mode gets its own slide.
- "think=on" — the model reasons before answering, and reasoning + answer SHARE one 8,192-token decode budget. Every number in this deck sits under that budget (next slide); the think=off deck is the clean, unconfounded read.
- "difficulty bins" (RQ0.5/0.6) — problems split into easy / medium / hard thirds, no-tools vs tool + nudge, to see whether the tool's advantage changes as problems get harder.
- A contamination re-run on disguised problems (sweep-6) left the no-tools baseline unchanged (footnote).

Read this first — every think=on number sits under a shared decode budget



Left: solve tool-use rate — neutral think=on collapses tool-calling vs think=off (Gemma-MoE-26B 100→23, Qwen3.5-9B 100→59); steering partly restores it. Right: solve truncation spikes under think=on. Consequence for everything that follows: the no-tools baseline often burns the whole 8,192-token budget reasoning and never answers, so availability gaps are inflated by baseline truncation — budget exhaustion, not ability.

Why the verdicts count direction — signed significance

A gap counts toward a YES only when the two arms' 95% intervals are disjoint AND the gap points the hypothesised way (the tool helps). Significant gaps pointing the other way are tallied separately as significant-AGAINST — they are findings, not noise.

- The case that makes this matter: on validate_plan, merely making the tool available moves Gemma-MoE-26B by -10pp — significant, and AGAINST the tool (it reasons instead of calling the tool — tool_selected ≈1% — and answers almost nothing).
- A sign-blind test would count that gap as evidence the tool 'has an effect' and read RQ0.3 as YES. The signed rule reads it as MIXED — which is what the data actually says.
- think=on verdicts are computed by this same rule at build time; only the think=off verdicts are locked. That the pattern (YES/YES/MIXED/YES) reproduces under reasoning mode is itself a finding.

RQ0.1 — validate_domain + validate_problem

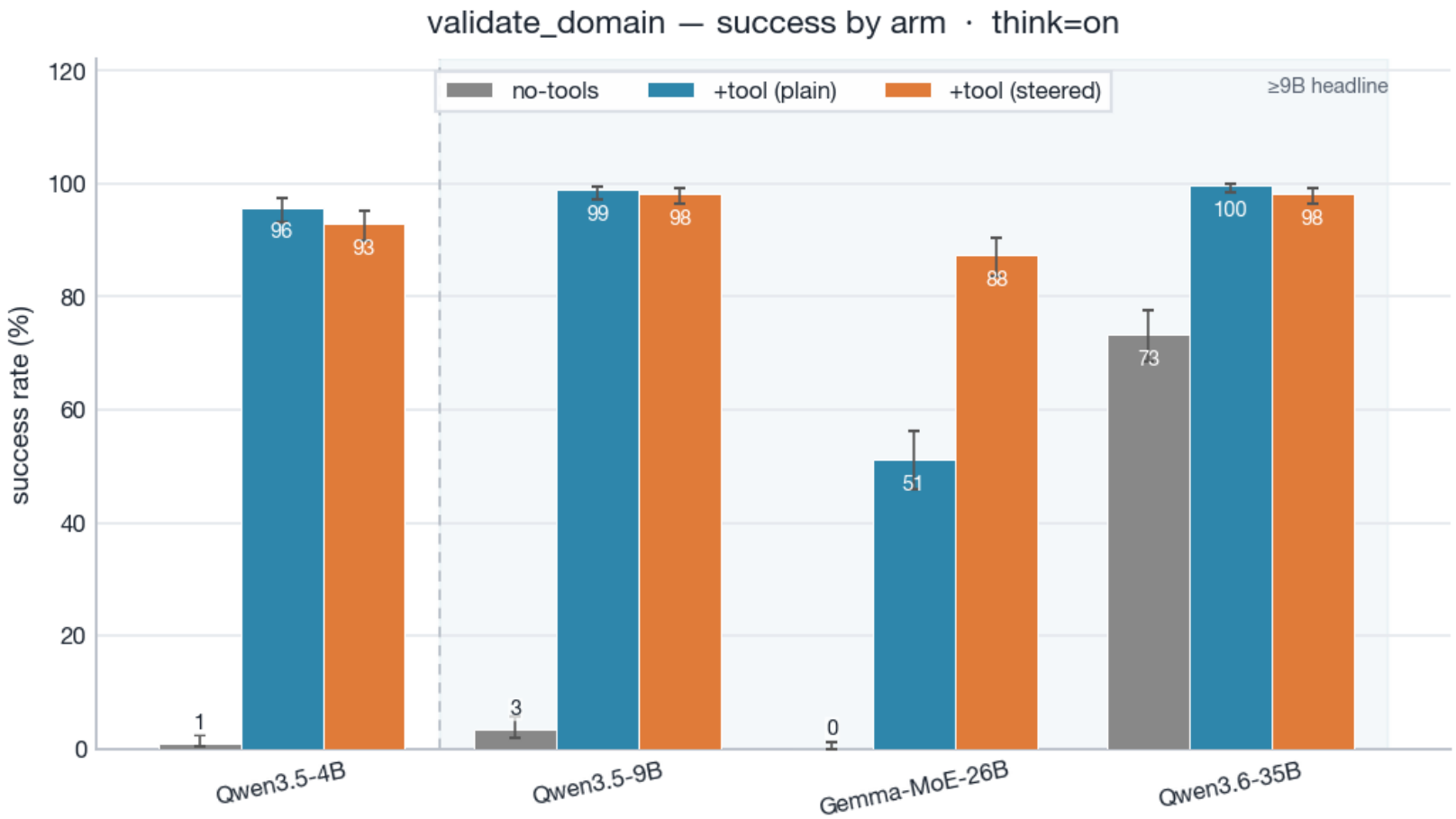
YES

Does giving the model a validation tool help it validate PDDL domains and problems? (validate_domain + validate_problem)

- Availability (validate_problem, ≥9B): gap +21…+74pp; 3/3 favorable-significant, 0/3 significant-against (95% CIs).
- Steering (validate_problem, ≥9B): gap -1…+29pp; 1/3 favorable-significant.

The no-tools baseline COLLAPSES under reasoning mode (validate_domain: 9B 26→3%, Gemma 78→0% vs think=off; 55–65% of baseline trials truncate), so the huge availability gaps are baseline-confounded — the honest read is that the tool arms are nearly immune to reasoning mode (≥9B steered 87–98%) while the unaided model is not. The 0.8B reversal from think=off disappears: its baseline is at the floor too.

RQ0.1 — Evidence: validate_domain success by arm

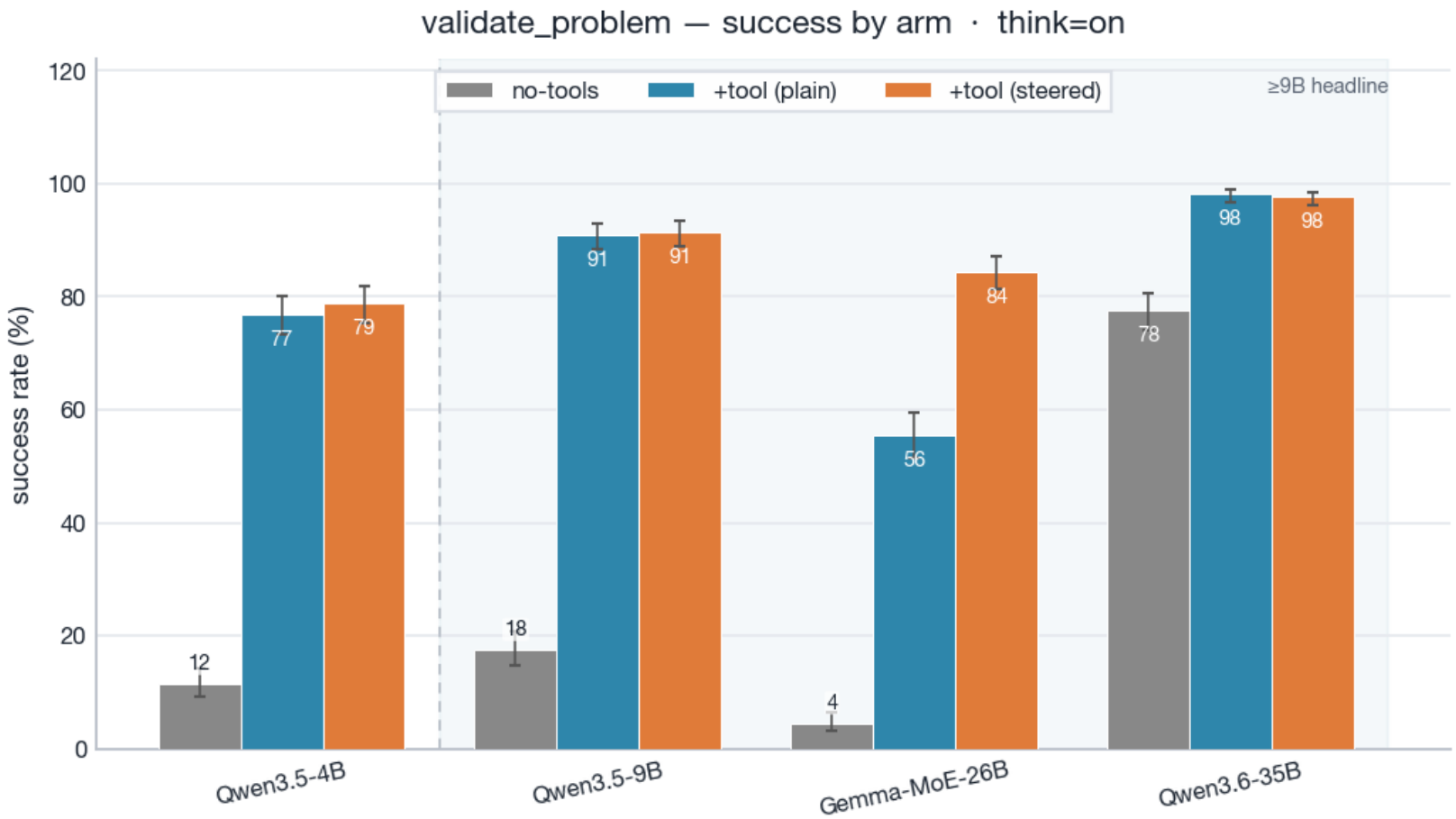


Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.1 — validate_domain success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	1 [0,2]	96 [93,97]	93 [90,95]	**+95***	-3
Qwen3.5-9B	3 [2,6]	99 [97,100]	98 [96,99]	**+96***	-1
Gemma-MoE-26B	0 [0,1]	51 [46,56]	88 [84,91]	**+51***	**+36***
Qwen3.6-35B	73 [69,78]	100 [98,100]	98 [96,99]	**+26***	-1

RQ0.1 — Evidence: validate_problem success by arm

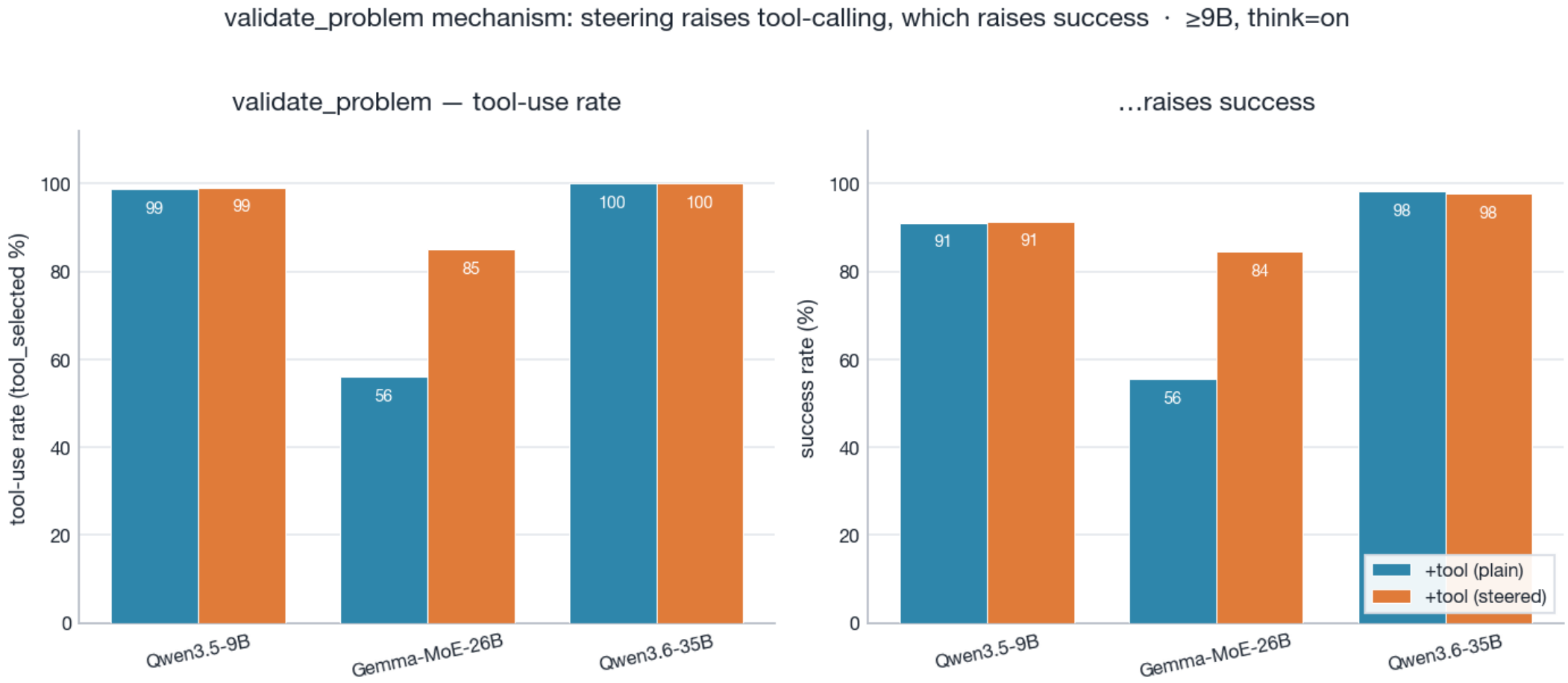


Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.1 — validate_problem success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	12 [9,14]	77 [73,80]	79 [75,82]	**+65***	+2
Qwen3.5-9B	18 [15,21]	91 [88,93]	91 [89,93]	**+74***	+0
Gemma-MoE-26B	4 [3,6]	56 [52,59]	84 [81,87]	**+51***	**+29***
Qwen3.6-35B	78 [74,81]	98 [97,99]	98 [96,99]	**+21***	-1

RQ0.1 — Mechanism: steering raises tool-calling → raises success



validate_problem, ≥9B. Left: tool-use rate (tool_selected) plain vs steered. Right: success. Where +tool(plain) under-calls the tool, steering raises tool-calling, and success rises with it — the tool's value is gated on the model actually invoking it.

RQ0.2 — solve

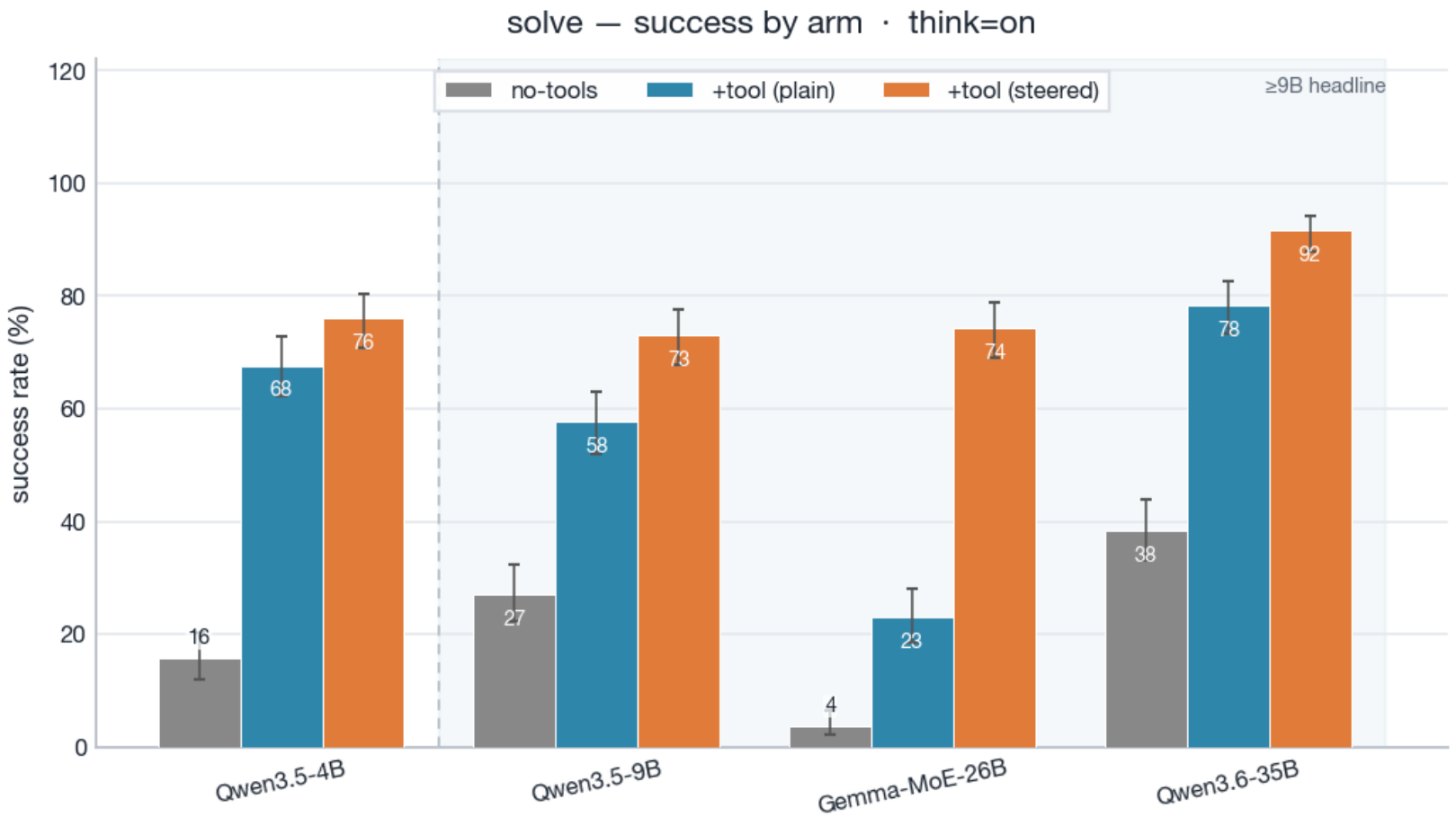
YES

Does giving the model a planning tool help it SOLVE PDDL problems? (solve)

- Availability (solve, ≥9B): gap +19…+40pp; 3/3 favorable-significant, 0/3 significant-against (95% CIs).
- Steering (solve, ≥9B): gap +13…+51pp; 3/3 favorable-significant.

solve is the one task where reasoning HELPS the unaided baseline (9B 11→27%, 35B 9→38% vs think=off). Tools still add +19…+40pp, and steering is now significant on all three models (Gemma 23→74%): under the shared budget, plain tool-calling is fragile — the model reasons instead of calling — and the nudge repairs it.

RQ0.2 — Evidence: solve success by arm

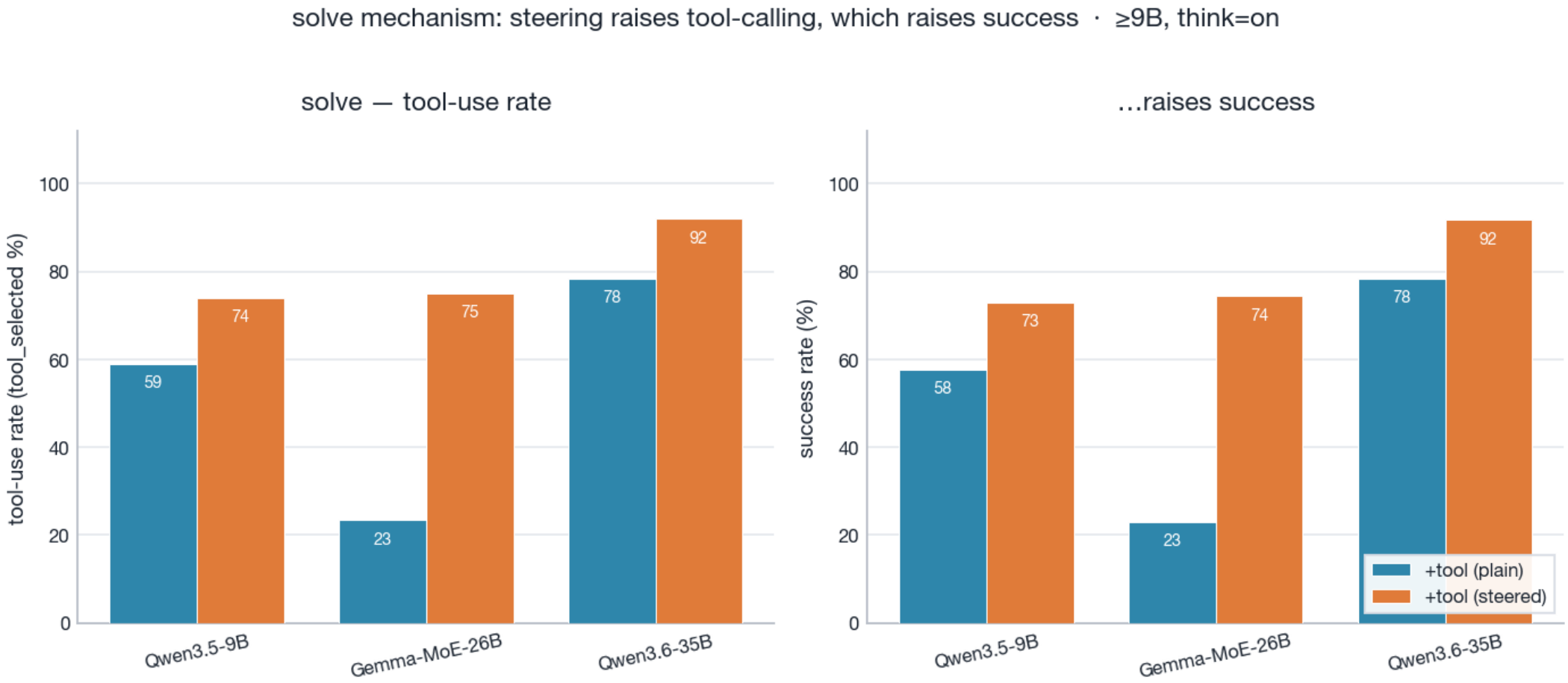


Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.2 — solve success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	16 [12,20]	68 [62,73]	76 [71,80]	**+52***	+8
Qwen3.5-9B	27 [22,32]	58 [52,63]	73 [68,78]	**+31***	**+15***
Gemma-MoE-26B	4 [2,6]	23 [19,28]	74 [69,79]	**+19***	**+51***
Qwen3.6-35B	38 [33,44]	78 [73,83]	92 [88,94]	**+40***	**+13***

RQ0.2 — Mechanism: steering raises tool-calling → raises success



solve, ≥9B. Left: tool-use rate (tool_selected) plain vs steered. Right: success. Where +tool(plain) under-calls the tool, steering raises tool-calling, and success rises with it — the tool's value is gated on the model actually invoking it.

RQ0.3 — validate_plan

MIXED

Does giving the model a validation tool help it CHECK whether a plan is valid? (validate_plan)

- Availability (validate_plan, ≥9B): gap -10…+44pp; 2/3 favorable-significant, 1/3 significant-against (95% CIs).
- Steering (validate_plan, ≥9B): gap +0…+43pp; 2/3 favorable-significant.

Still MIXED, more extreme: Gemma's plain arm collapses to 0.6% (tool_selected ≈1% — it reasons instead of calling; gate slide next). Steering recovers it to 44% — still BELOW its own think=off no-tools 88%. The strong unaided baseline that made this RQ mixed at think=off is itself destroyed by reasoning (9B 80→21%, Gemma 88→10%); only 35B's survives (85%) and it is the one favourable-but-small cell.

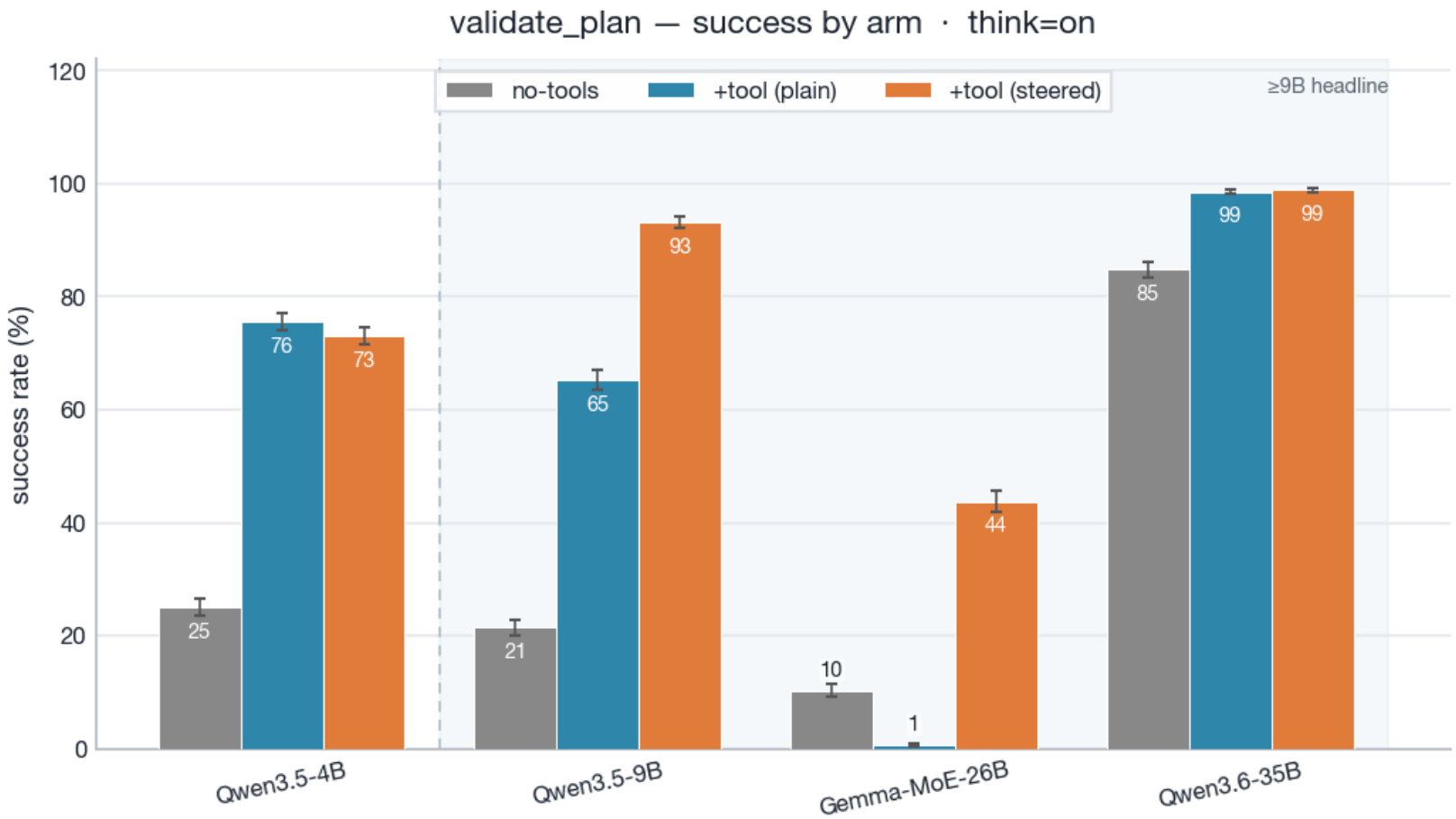
Reading guide — the low +tool(plain) bar is silence, not error

On validate_plan the +tool(plain) arm often produces NO verdict at all — the model fails to call the tool and answers nothing. When it does answer, it is near-perfect:

- Gemma-MoE-26B — raw success 1%, but it only produces a verdict on 1% of trials; when it DOES answer, accuracy is 100% (false-positives: 0).
- Qwen3.5-9B — raw success 65%, but it only produces a verdict on 68% of trials; when it DOES answer, accuracy is 96% (false-positives: 15).

→ So the collapse is a tool-CALLING failure, not a verdict failure — and steering, which raises tool-calling, repairs it (mechanism slide after the evidence).

RQ0.3 — Evidence: validate_plan success by arm

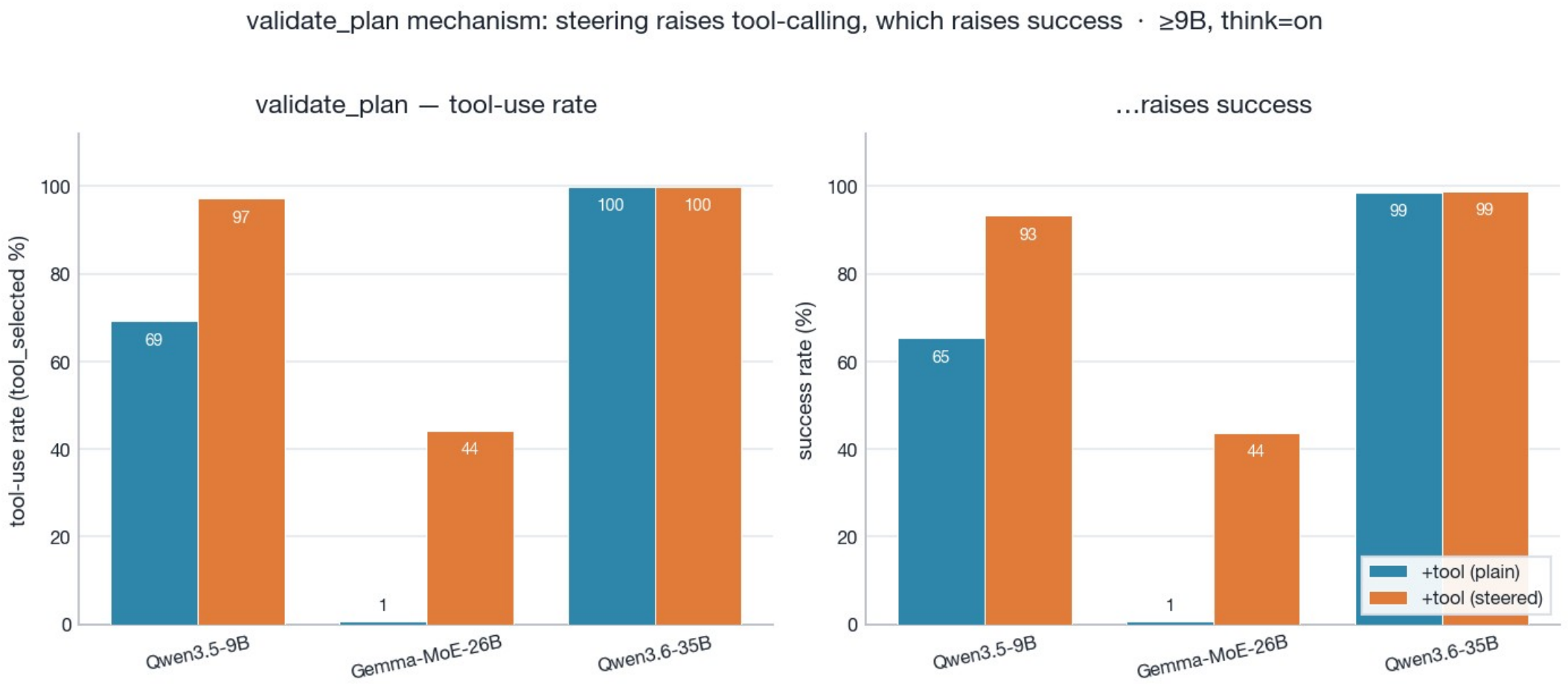


Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.3 — validate_plan success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	25 [23,27]	76 [74,77]	73 [71,75]	**+51***	-3
Qwen3.5-9B	21 [20,23]	65 [64,67]	93 [92,94]	**+44***	**+28***
Gemma-MoE-26B	10 [9,11]	1 [0,1]	44 [42,46]	**-10***	**+43***
Qwen3.6-35B	85 [83,86]	99 [98,99]	99 [98,99]	**+14***	+0

RQ0.3 — Mechanism: steering raises tool-calling → raises success



validate_plan, ≥9B. Left: tool-use rate (tool_selected) plain vs steered. Right: success. Where +tool(plain) under-calls the tool, steering raises tool-calling, and success rises with it — the tool's value is gated on the model actually invoking it.

RQ0.4 — simulate

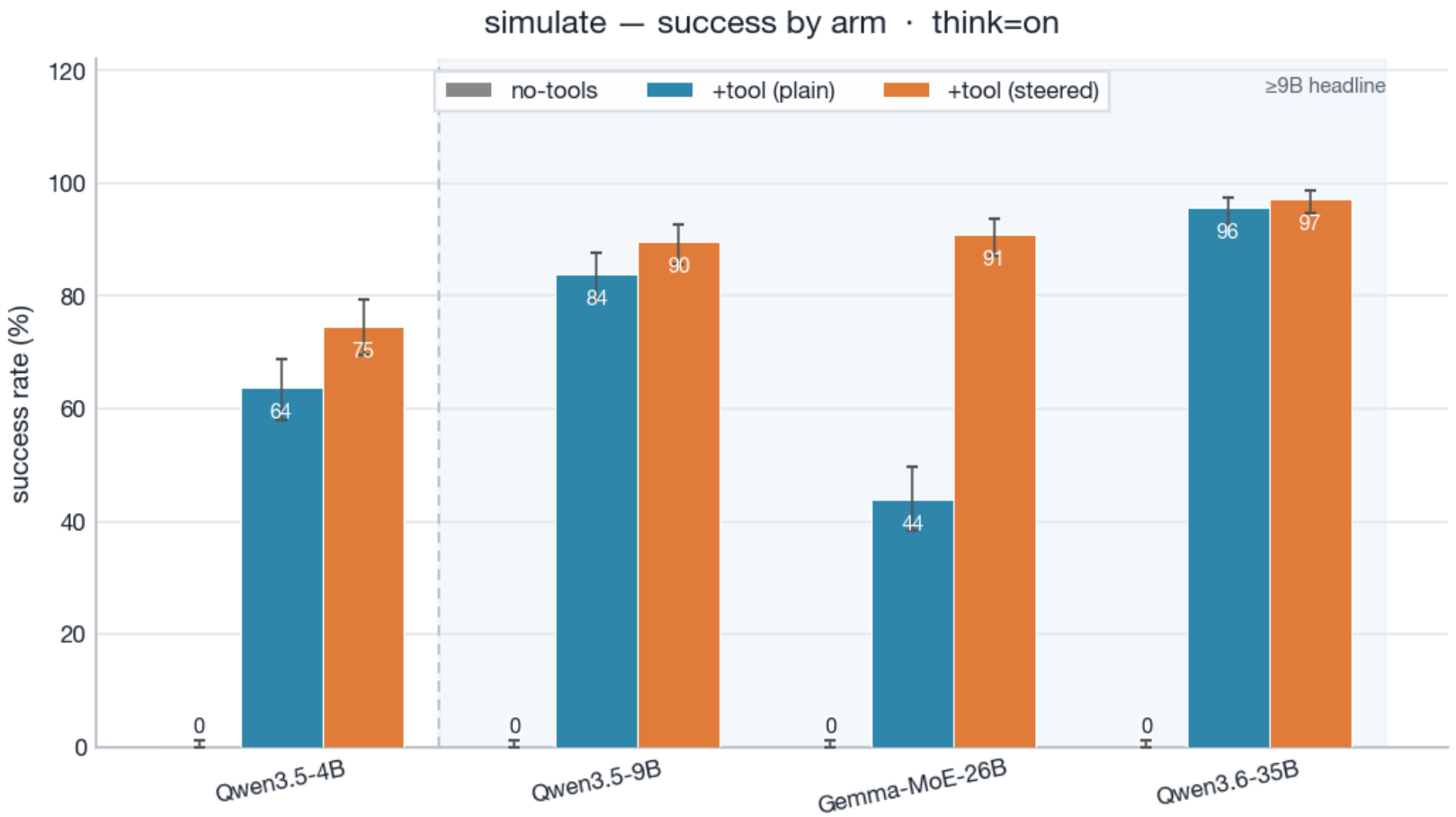
YES

Does giving the model a simulation tool help it TRACK state (simulate a plan)? (simulate)

- Availability (simulate, ≥9B): gap +44…+96pp; 3/3 favorable-significant, 0/3 significant-against (95% CIs).
- Steering (simulate, ≥9B): gap +2…+47pp; 1/3 favorable-significant.

Identical to think=off in kind: no-tools is 0% everywhere — reasoning does not buy state-tracking, it just burns the budget (83% truncation). +tool reaches 44–96% plain; steering rescues Gemma (44→91%).

RQ0.4 — Evidence: simulate success by arm

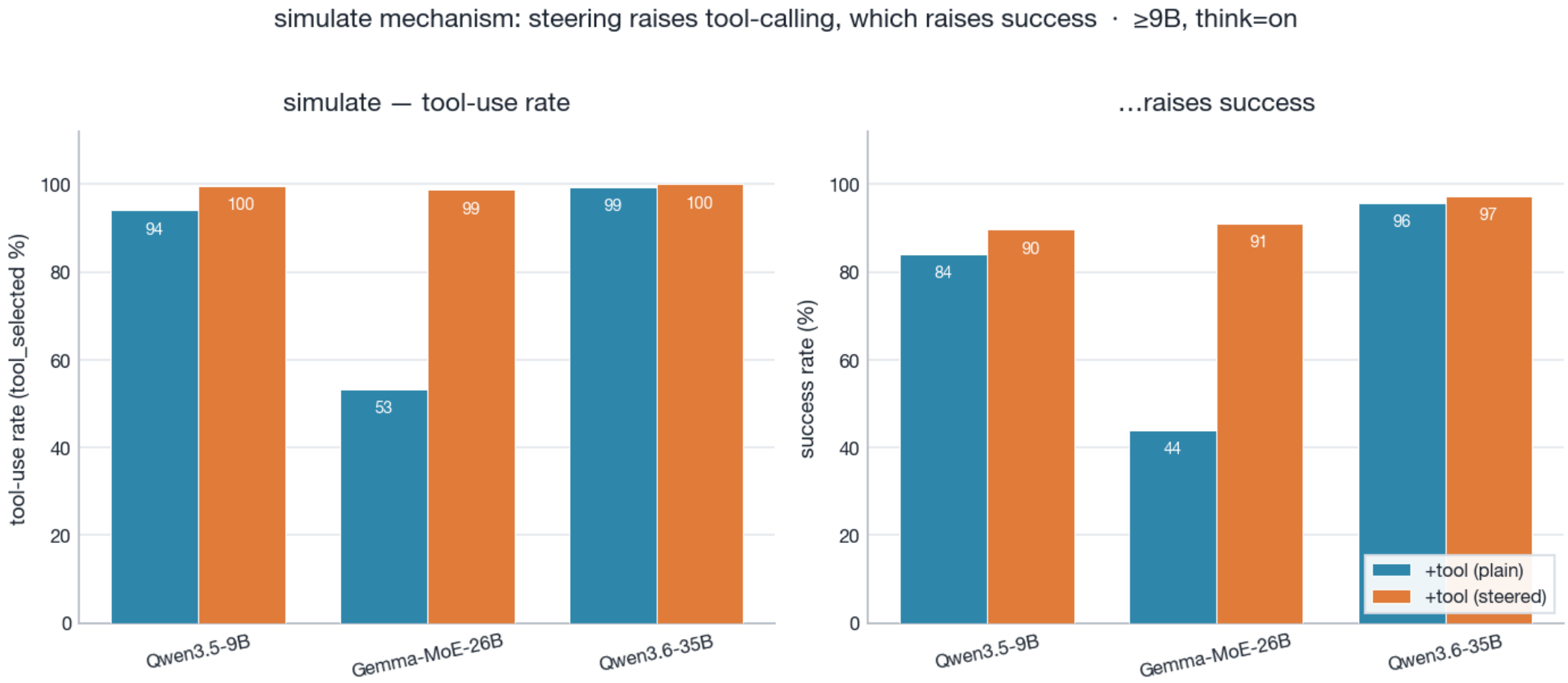


Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.4 — simulate success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	0 [0,1]	64 [58,69]	75 [69,79]	**+64***	**+11***
Qwen3.5-9B	0 [0,1]	84 [79,88]	90 [86,93]	**+84***	+6
Gemma-MoE-26B	0 [0,1]	44 [38,50]	91 [87,94]	**+44***	**+47***
Qwen3.6-35B	0 [0,1]	96 [93,97]	97 [95,99]	**+96***	+2

RQ0.4 — Mechanism: steering raises tool-calling → raises success



simulate, ≥9B. Left: tool-use rate (tool_selected) plain vs steered. Right: success. Where +tool(plain) under-calls the tool, steering raises tool-calling, and success rises with it — the tool's value is gated on the model actually invoking it.

Small-model record — Qwen3.5-0.8B under think=on

task	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
solve	0 [0,1]	12 [9,16]	23 [19,28]	**+12***	**+11***
validate_domain	0 [0,2]	84 [80,88]	89 [85,91]	**+84***	+4
validate_problem	0 [0,1]	14 [11,17]	20 [17,24]	**+14***	**+6***
validate_plan	0 [0,0]	17 [16,19]	19 [17,20]	**+17***	+1
simulate	0 [0,1]	5 [3,8]	2 [1,5]	**+5***	-3

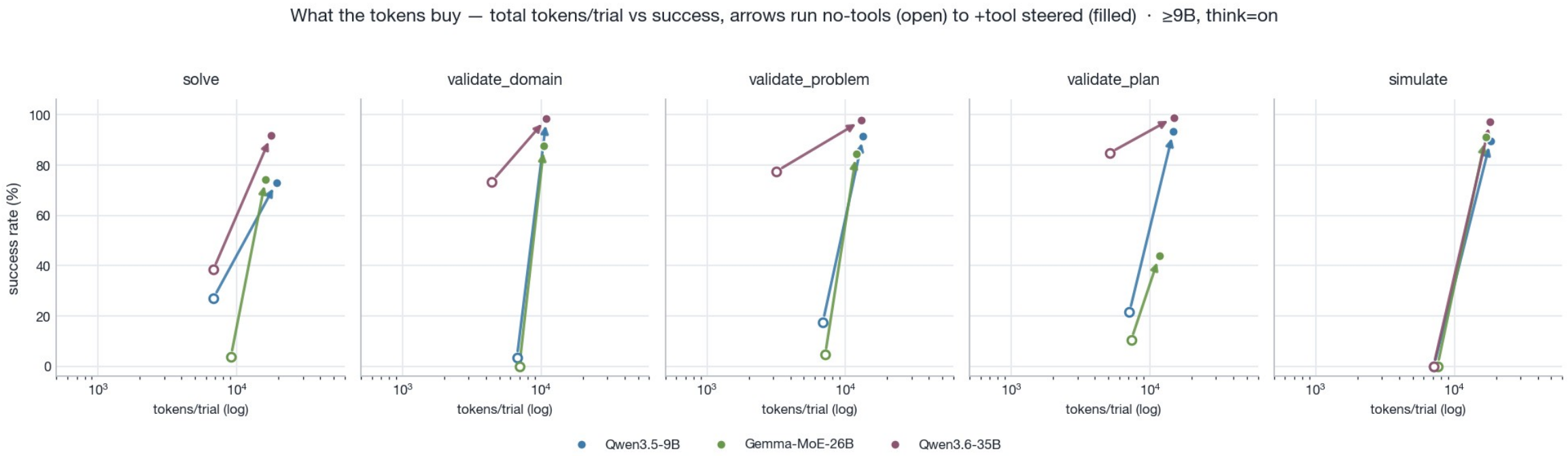
What do tools cost? — token consumption

Token cost = TOTAL tokens a trial consumes — input (prompt) + output (completion), summed across the model's turns. The whole bill, not just generated text. Scope: ≥9B, think=on.

- Under think=on tools cost only ~2× per trial — not because tools got cheaper, but because the no-tools baseline now burns ~5–6k output tokens reasoning (and usually truncates). The next figure holds cost and success in one view.
- Quality-adjusted efficiency is cost-of-pass = total tokens ÷ correct answers: every token burned on a failed attempt is charged to the successes. LOWER is better.

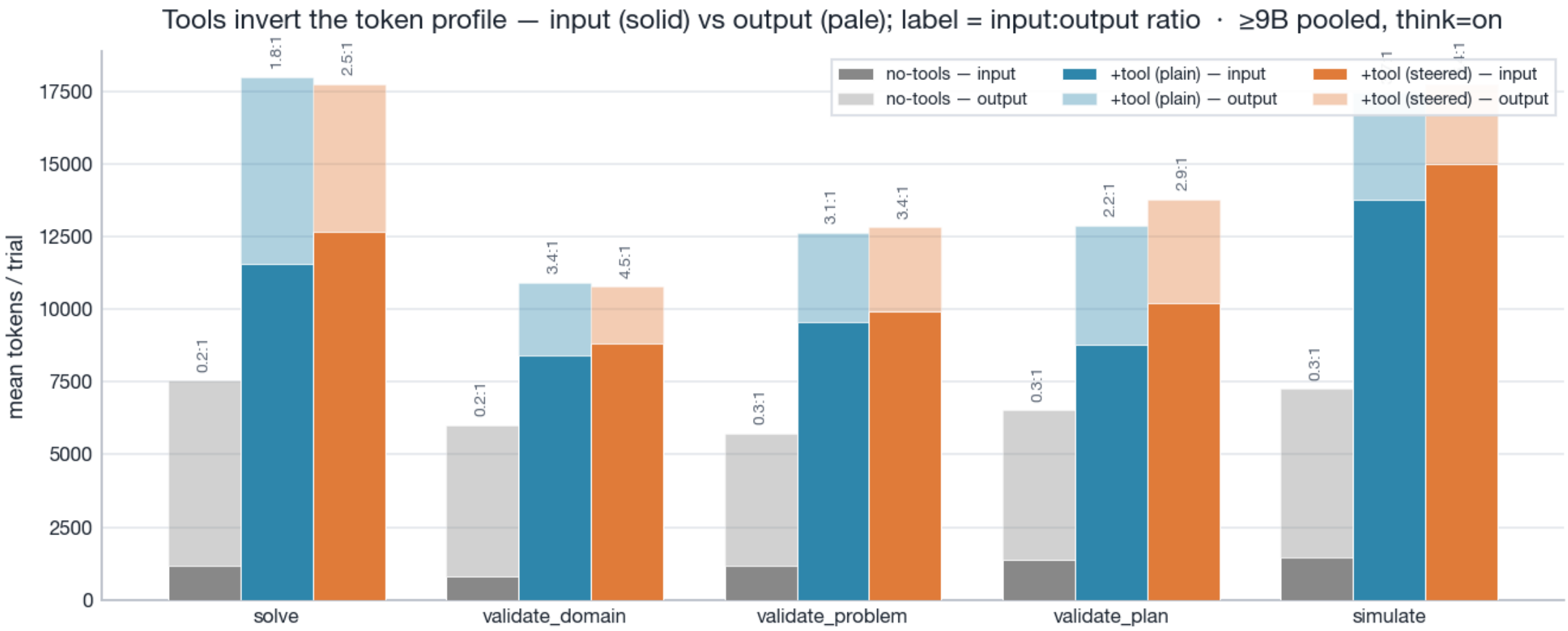
→ Caveats: (1) prefix caching (~90% on this roster) makes the re-sent tool INPUT cheap in server compute — but it is still real context-budget consumption, and tool OUTPUTS across turns are novel/uncached; the raw total is the honest accounting number. (2) Output is right-censored at the 8,192-token cap with arm-dependent truncation (evidence slide at the end of this section). (3) Under think=on, completion = REASONING + answer in one number — the runner logs no thinking/answer split (vLLM strips the trace server-side), so output tokens cannot be read as answer length; totals remain the honest bill. The completion-only lens stays in the backup.

What the tokens buy — pay ~2× per trial, buy +14…+97pp



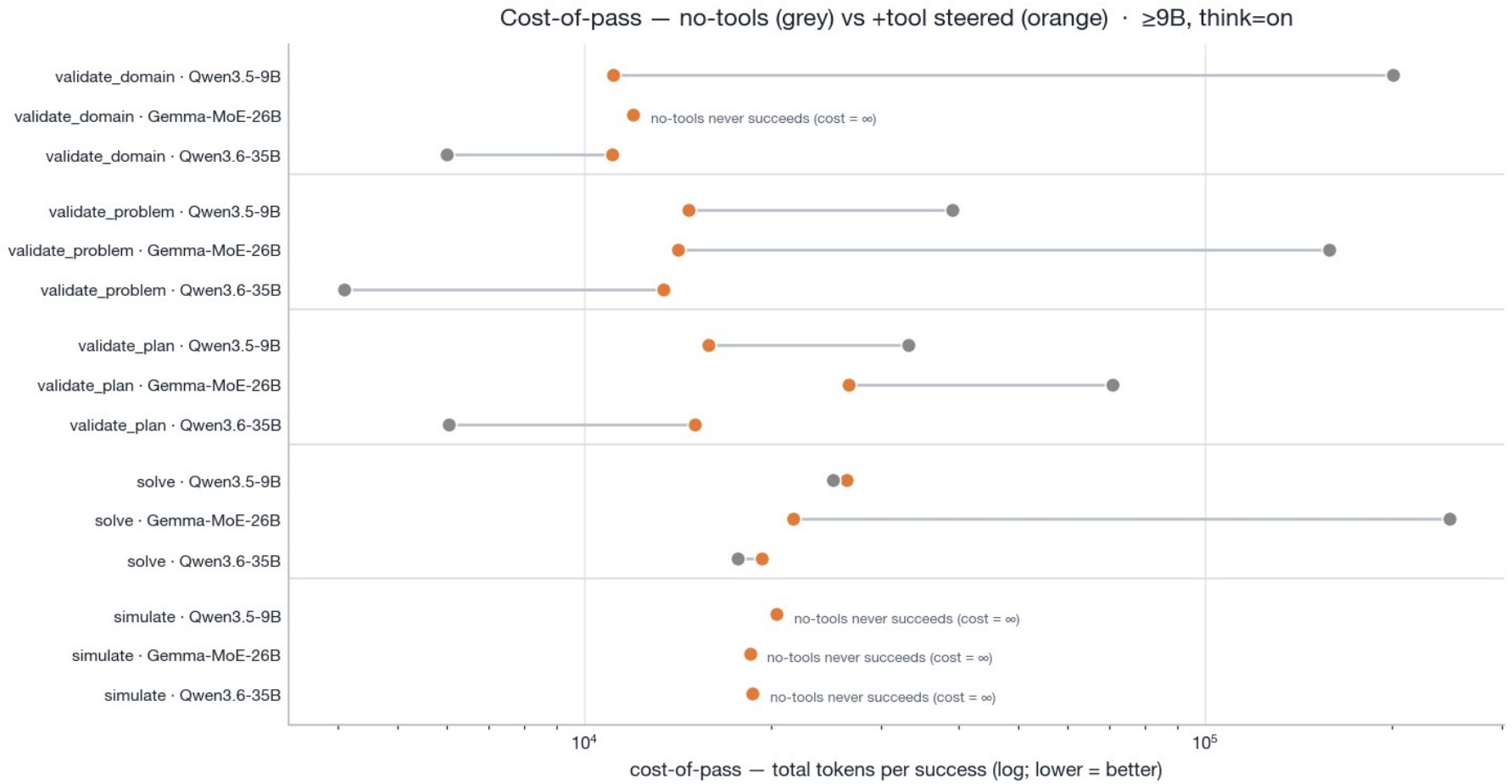
Each arrow is one ≥9B model on one task, from no-tools ○ to +tool(steered) ● — x = mean total tokens/trial (log), y = success. Arrows are short on the x-axis (the baseline already burns its budget reasoning) and long on the y-axis: under think=on the tool buys large success gaps at a modest ~2× per-trial premium. Per-model tables in the backup.

Tools invert the token profile — the cost is input, not output



Mean input (solid) vs output (pale) tokens per trial, ≥9B pooled. no-tools is output-heavy (~0.2:1 input:output — the reasoning trace is all output); the tool arms are input-heavy (~3–5:1) because tool schemas + tool outputs are re-fed every turn. That re-fed input is the real token cost of tools — an output-only comparison hides it entirely, which is why the headline metric is the total.

Cost-of-pass — under think=on the regime is per-MODEL, and the tool wins almost everywhere



Total tokens per success (log; lower = better), no-tools ○ vs +tool(steered) ●, bootstrap point estimates. The think=off task-regime split dissolves into a per-MODEL one: where reasoning drowns the baseline (9B validate_domain 200k→11k, Gemma ∞→12k tokens/success) the tool is far cheaper or the only producer; only Qwen3.6-35B — whose baseline survives the budget — still pays a tool premium on validate_* (e.g. 6k→15k). simulate is ∞ without the tool for every model.

Why efficiency moves — (tokens/trial) ÷ (more often right) = cost-per-success

task	model	tokens/trial ×	more often right ×	= cost-per-success ×
solve	Qwen3.5-4B	2.3× costlier	4.9× more right	0.5× cheaper
	Qwen3.5-9B	2.9× costlier	2.7× more right	1.1× costlier
	Gemma-MoE-26B	1.8× costlier	20.3× more right	0.09× cheaper
	Qwen3.6-35B	2.6× costlier	2.4× more right	1.1× costlier
validate_domain	Qwen3.5-4B	1.6× costlier	111.7× more right	0.01× cheaper
	Qwen3.5-9B	1.6× costlier	29.5× more right	0.06× cheaper
	Gemma-MoE-26B	—	—	—
	Qwen3.6-35B	2.5× costlier	1.3× more right	1.8× costlier
validate_problem	Qwen3.5-4B	2.6× costlier	6.9× more right	0.4× cheaper
	Qwen3.5-9B	2.0× costlier	5.2× more right	0.4× cheaper
	Gemma-MoE-26B	1.7× costlier	18.8× more right	0.09× cheaper
	Qwen3.6-35B	4.1× costlier	1.3× more right	3.3× costlier
validate_plan	Qwen3.5-4B	3.5× costlier	2.9× more right	1.2× costlier
	Qwen3.5-9B	2.1× costlier	4.4× more right	0.5× cheaper
	Gemma-MoE-26B	1.6× costlier	4.2× more right	0.4× cheaper
	Qwen3.6-35B	2.9× costlier	1.2× more right	2.5× costlier

Are the token numbers comparable? — truncation, turns, latency proxy

task	arm	truncated %	mean turns	output tok/trial	total tok/trial
solve	no-tools	55%	1.0	6.4k	7.5k
	+tool (plain)	60%	2.1	6.4k	18k
	+tool (steered)	40%	2.3	5.1k	18k
validate_domain	no-tools	65%	1.0	5.2k	6.0k
	+tool (plain)	17%	1.9	2.5k	11k
	+tool (steered)	6%	2.0	2.0k	11k
validate_problem	no-tools	59%	1.0	4.6k	5.7k
	+tool (plain)	16%	1.9	3.1k	13k
	+tool (steered)	10%	2.0	2.9k	13k
validate_plan	no-tools	60%	1.0	5.1k	6.5k
	+tool (plain)	34%	1.6	4.1k	13k
	+tool (steered)	25%	1.8	3.6k	14k
simulate	no-tools	83%	1.0	5.8k	7.2k
	+tool (plain)	80%	2.0	3.7k	17k
	+tool (steered)	76%	2.1	2.8k	18k

RQ0.5 — does the advantage track difficulty?

YES

Does the planning tool's advantage CHANGE as instances get harder (longer reference / plan length)?

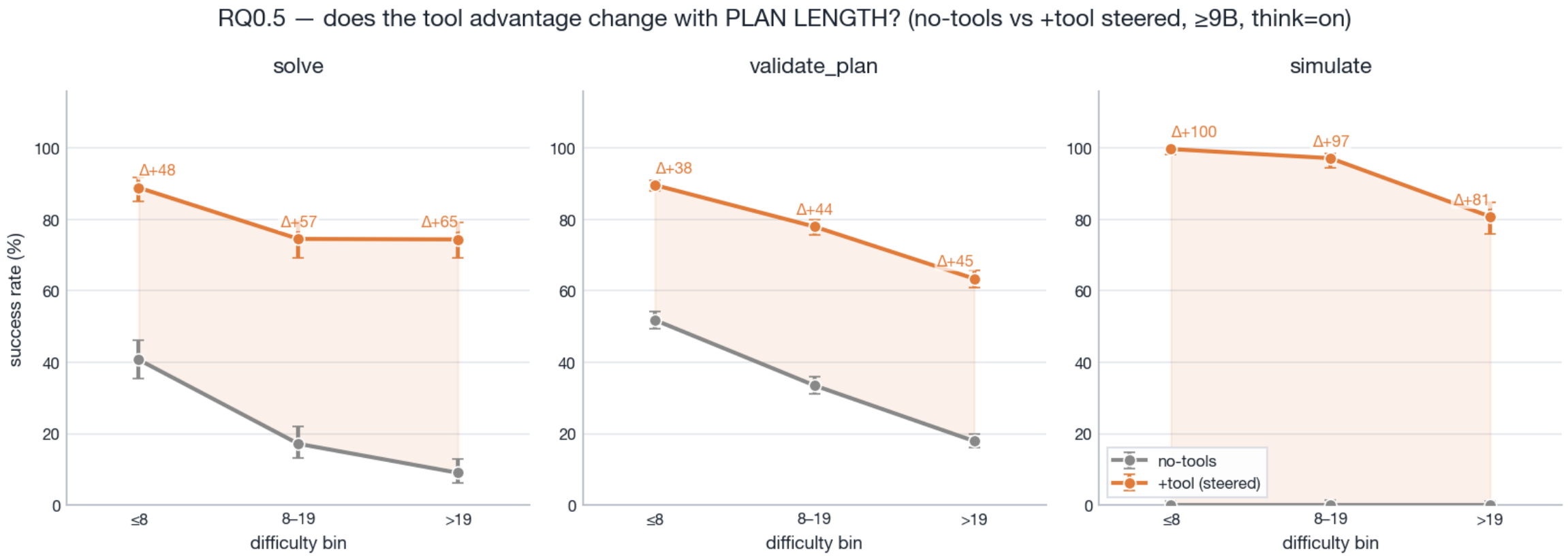
YES — and under think=on the headroom case moves to SOLVE.

Headroom case (solve) gap by bin: ≤8: Δ+48 → 8–19: Δ+57 → >19: Δ+65.

- solve is now headroom-gated: reasoning gives the unaided baseline a real solve rate on short plans (41%) that fades with length (17→9%), while the tool arm holds (89→74%) — the gap widens +48→+65pp. Open-ended reasoning runs out of budget exactly where plans get long; the tool does not.
- validate_plan widens mildly (+38→+45pp); simulate stays tool-arm-only (no-tools floored at 0%).

Phase-2 is headroom-gated: an advantage can only be seen to CHANGE where both arms have room to move. ≥9B, think=on, no-tools vs +tool(steered).

RQ0.5 — Evidence



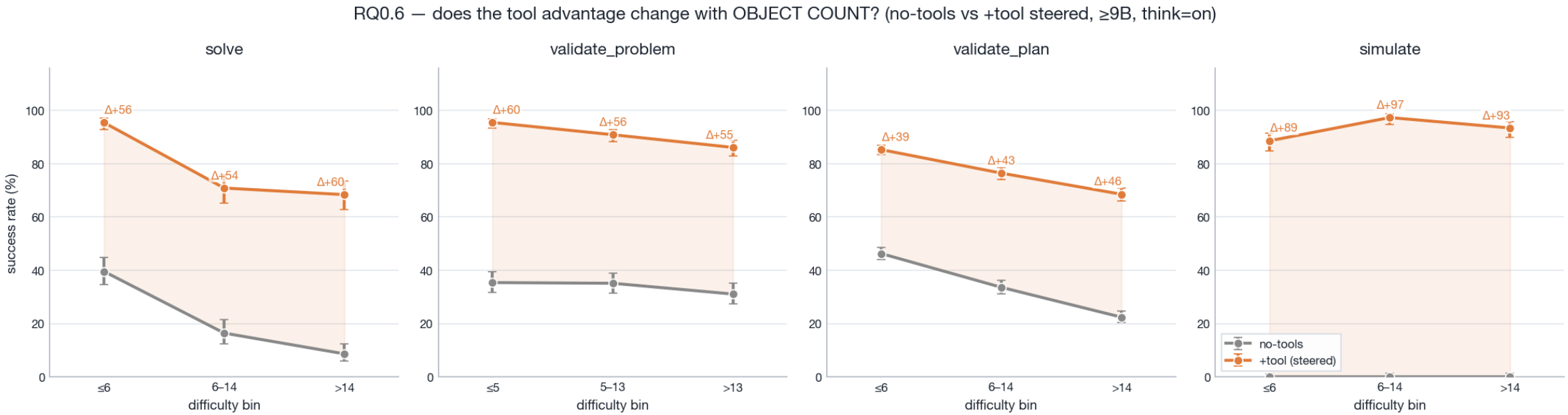
Per difficulty bin: no-tools (grey) vs +tool steered (orange), Wilson 95% CIs; Δ = tool − no-tools over each bin (shaded = the advantage).

RQ0.5 — difficulty-binned success (no-tools vs +tool steered)

task	bin	no-tools	+tool(steered)	Δ (pp)	n/arm
solve	≤8	41 [36,46]	89 [85,92]	+48	324
	8–19	17 [13,22]	75 [69,79]	+57	279
	>19	9 [6,13]	74 [69,79]	+65	297
validate_plan	≤8	52 [49,54]	90 [88,91]	+38	1620
	8–19	34 [31,36]	78 [76,80]	+44	1395
	>19	18 [16,20]	63 [61,66]	+45	1485
simulate	≤8	0 [0,1]	100 [98,100]	+100	324
	8–19	0 [0,1]	97 [94,99]	+97	279
	>19	0 [0,1]	81 [76,85]	+81	297

RQ0.6 — object count does not move the advantage

NO



Headroom case (validate_problem): gap ≤5: Δ+60 → 5–13: Δ+56 → >13: Δ+55pp — essentially flat; the other tasks are also flat (no-tools floored or already separated). Object count is not a difficulty axis the tool's advantage tracks. Full bin table in the backup.

Robustness footnote — contamination probe (sweep-6)

- A separate anonymised-corpus sweep (sweep-6) re-ran the matrix on structurally-identical domains with every surface name renamed, to test memorisation.
- Headline (think=off): the clean no-tools-neutral probe is near-null — Δ(canonical − anon) success ≤1.3pp mean |Δ|, zero CI-disjoint task cells. No broad train-set contamination of the pure-model baseline where it has headroom.
- think=on caveat: the probe's only CI-disjoint cells (validate_plan, think=on) were a TOKENISATION artifact — anonymised prompts ran ~5% longer, so more trials truncated under the shared budget; success-given-completion was equal. Not memorisation — but a live demonstration of how budget exhaustion, not ability, moves think=on numbers.

Out of scope (phase-3)

- Cross-benchmark generalisation (PlanBench) and comparison to published SOTA formalizer baselines (e.g. Huang & Zhang, ACL 2025) are deliberately OUT OF SCOPE for this single-tool-use deck.
- This deck answers: within our 5-model × 5-task matrix, does giving the model a planning/validation tool (and steering it to use the tool) raise success, and does that advantage track difficulty?

Backup — detailed tables

The slides that follow hold the full per-model numbers behind the token section:

- total tokens/trial (input+output) per task × model × arm, with cost multiples vs no-tools;
- cost-of-pass (total tokens per success) with bootstrap 95% CIs;
- the secondary completion-only generation-cost lens — output tokens over successes only. It excludes the re-fed tool input (the dominant tool cost) and failed-attempt tokens, so it flatters tools; it is a generation-length diagnostic, never the consumption headline;
- the RQ0.6 difficulty-bin table.

Backup — total tokens/trial (input+output) · ≥4B, think=on · 1/2

task	model	no-tools	+tool(plain)	+tool(steered)
solve	Qwen3.5-4B	9.0k (1.2k+7.8k)	23k (18k+5.3k) 2.5× costlier	21k (16k+4.5k) 2.3× costlier
	Qwen3.5-9B	6.8k (1.2k+5.6k)	20k (14k+6.4k) 3.0× costlier	19k (14k+5.1k) 2.9× costlier
	Gemma-MoE-26B	9.1k (1.2k+7.8k)	13k (6.3k+7.1k) 1.5× costlier	16k (10k+5.8k) 1.8× costlier
	Qwen3.6-35B	6.8k (1.2k+5.6k)	20k (14k+5.8k) 3.0× costlier	18k (13k+4.3k) 2.6× costlier
validate_domain	Qwen3.5-4B	6.9k (792+6.1k)	11k (9.2k+1.3k) 1.5× costlier	11k (9.3k+1.6k) 1.6× costlier
	Qwen3.5-9B	6.7k (792+5.9k)	12k (10k+1.6k) 1.8× costlier	11k (9.4k+1.5k) 1.6× costlier
	Gemma-MoE-26B	7.0k (817+6.1k)	10k (5.8k+4.4k) 1.5× costlier	10k (7.7k+2.8k) 1.5× costlier
	Qwen3.6-35B	4.4k (792+3.6k)	11k (9.2k+1.5k) 2.4× costlier	11k (9.3k+1.6k) 2.5× costlier
validate_problem	Qwen3.5-4B	7.0k (1.1k+5.8k)	18k (14k+3.9k) 2.6× costlier	18k (14k+3.9k) 2.6× costlier
	Qwen3.5-9B	6.9k (1.1k+5.7k)	14k (11k+2.5k) 2.0× costlier	13k (11k+2.4k) 2.0× costlier
	Gemma-MoE-26B	7.1k (1.2k+5.9k)	11k (6.9k+4.4k) 1.6× costlier	12k (8.3k+3.6k) 1.7× costlier
	Qwen3.6-35B	3.2k (1.1k+2.0k)	13k (10k+2.4k) 4.0× costlier	13k (10k+2.7k) 4.1× costlier

Backup — total tokens/trial (input+output) · ≥4B, think=on · 2/2

task	model	no-tools	+tool(plain)	+tool(steered)
validate_plan	Qwen3.5-4B	7.0k (1.4k+5.6k)	24k (19k+4.8k) 3.4× costlier	24k (20k+4.5k) 3.5× costlier
	Qwen3.5-9B	7.1k (1.4k+5.8k)	14k (10k+4.0k) 2.0× costlier	15k (12k+2.7k) 2.1× costlier
	Gemma-MoE-26B	7.3k (1.4k+5.9k)	9.7k (4.3k+5.4k) 1.3× costlier	12k (6.6k+5.1k) 1.6× costlier
	Qwen3.6-35B	5.1k (1.4k+3.8k)	15k (12k+2.8k) 2.9× costlier	15k (12k+2.9k) 2.9× costlier
simulate	Qwen3.5-4B	7.5k (1.4k+6.1k)	27k (23k+4.1k) 3.5× costlier	23k (19k+3.5k) 3.0× costlier
	Qwen3.5-9B	7.1k (1.4k+5.7k)	18k (15k+2.8k) 2.5× costlier	18k (16k+2.5k) 2.6× costlier
	Gemma-MoE-26B	7.6k (1.5k+6.1k)	16k (11k+5.1k) 2.1× costlier	17k (14k+2.8k) 2.2× costlier
	Qwen3.6-35B	7.1k (1.4k+5.6k)	19k (15k+3.2k) 2.6× costlier	18k (15k+3.0k) 2.6× costlier

Backup — cost-of-pass, tokens per success [bootstrap 95% CI] · ≥4B, think=on · 1/2

task	model	no-tools	+tool(plain)	+tool(steered)
solve	Qwen3.5-4B	57k [45k,78k]	34k [31k,37k] 0.6× cheaper	27k [25k,30k] 0.5× cheaper
	Qwen3.5-9B	25k [20k,33k]	35k [33k,39k] 1.4× costlier	26k [25k,28k] 1.1× costlier
	Gemma-MoE-26B	247k [152k,536k]	58k [49k,72k] 0.2× cheaper	22k [20k,23k] 0.09× cheaper
	Qwen3.6-35B	18k [15k,21k]	26k [24k,27k] 1.5× costlier	19k [18k,20k] 1.1× costlier
validate_domain	Qwen3.5-4B	829k [355k,2496k]	11k [11k,11k] 0.01× cheaper	12k [11k,12k] 0.01× cheaper
	Qwen3.5-9B	200k [126k,404k]	12k [11k,13k] 0.06× cheaper	11k [11k,11k] 0.06× cheaper
	Gemma-MoE-26B	— (0 succ)	20k [18k,22k]	12k [11k,13k]
	Qwen3.6-35B	6.0k [5.5k,6.5k]	11k [11k,11k] 1.8× costlier	11k [11k,11k] 1.8× costlier
validate_problem	Qwen3.5-4B	61k [49k,78k]	24k [22k,25k] 0.4× cheaper	23k [22k,25k] 0.4× cheaper
	Qwen3.5-9B	39k [33k,47k]	15k [15k,16k] 0.4× cheaper	15k [14k,15k] 0.4× cheaper
	Gemma-MoE-26B	158k [111k,253k]	20k [19k,22k] 0.1× cheaper	14k [14k,15k] 0.09× cheaper
	Qwen3.6-35B	4.1k [3.9k,4.4k]	13k [13k,13k] 3.2× costlier	13k [13k,14k] 3.3× costlier

Backup — cost-of-pass, tokens per success [bootstrap 95% CI] · ≥4B, think=on · 2/2

task	model	no-tools	+tool(plain)	+tool(steered)
validate_plan	Qwen3.5-4B	28k [26k,30k]	31k [30k,32k] 1.1× costlier	33k [32k,34k] 1.2× costlier
	Qwen3.5-9B	33k [31k,36k]	22k [21k,22k] 0.7× cheaper	16k [16k,16k] 0.5× cheaper
	Gemma-MoE-26B	71k [64k,79k]	1525k [1030k,2633k] 21.5× costlier	27k [26k,28k] 0.4× cheaper
	Qwen3.6-35B	6.0k [5.9k,6.2k]	15k [15k,15k] 2.5× costlier	15k [15k,15k] 2.5× costlier
simulate	Qwen3.5-4B	— (0 succ)	42k [37k,48k]	30k [27k,34k]
	Qwen3.5-9B	— (0 succ)	21k [20k,23k]	20k [19k,22k]
	Gemma-MoE-26B	— (0 succ)	36k [32k,41k]	18k [18k,19k]
	Qwen3.6-35B	— (0 succ)	19k [19k,20k]	19k [18k,19k]

Backup — completion-only output tokens per success (secondary, tool-flattering) · 1/2

task	model	no-tools	+tool(plain)	+tool(steered)
solve	Qwen3.5-4B	5.8k [5.3k,6.3k]	4.0k [3.7k,4.3k]	3.6k [3.4k,3.9k]
	Qwen3.5-9B	2.9k [2.4k,3.4k]	5.0k [4.6k,5.4k]	4.0k [3.7k,4.3k]
	Gemma-MoE-26B	7.7k [6.7k,8.2k]	5.4k [4.7k,6.0k]	5.0k [4.6k,5.4k]
	Qwen3.6-35B	4.0k [3.7k,4.3k]	5.1k [4.7k,5.4k]	3.9k [3.7k,4.2k]
validate_domain	Qwen3.5-4B	4.8k [4.0k,5.7k]	1.3k [1.2k,1.4k]	1.5k [1.4k,1.6k]
	Qwen3.5-9B	4.2k [3.4k,4.9k]	1.6k [1.5k,1.7k]	1.4k [1.3k,1.6k]
	Gemma-MoE-26B	— (0 succ)	2.8k [2.6k,3.0k]	2.3k [2.2k,2.5k]
	Qwen3.6-35B	3.3k [3.2k,3.4k]	1.5k [1.4k,1.6k]	1.5k [1.5k,1.6k]
validate_problem	Qwen3.5-4B	3.6k [3.3k,3.8k]	3.2k [2.9k,3.5k]	3.1k [2.9k,3.4k]
	Qwen3.5-9B	3.7k [3.5k,4.0k]	2.2k [2.1k,2.4k]	2.2k [2.0k,2.3k]
	Gemma-MoE-26B	1.7k [1.4k,2.0k]	3.3k [3.1k,3.5k]	3.2k [3.0k,3.3k]
	Qwen3.6-35B	1.9k [1.9k,2.0k]	2.4k [2.3k,2.5k]	2.7k [2.5k,2.8k]

Backup — completion-only output tokens per success (secondary, tool-flattering) · 2/2

task	model	no-tools	+tool(plain)	+tool(steered)
validate_plan	Qwen3.5-4B	4.2k [4.1k,4.3k]	3.9k [3.8k,4.0k]	3.5k [3.4k,3.6k]
	Qwen3.5-9B	4.3k [4.2k,4.4k]	3.6k [3.5k,3.7k]	2.5k [2.4k,2.6k]
	Gemma-MoE-26B	3.8k [3.7k,4.0k]	2.1k [1.8k,2.3k]	3.9k [3.8k,4.0k]
	Qwen3.6-35B	3.4k [3.4k,3.5k]	2.8k [2.8k,2.9k]	2.9k [2.9k,3.0k]
simulate	Qwen3.5-4B	— (0 succ)	2.6k [2.3k,2.8k]	2.6k [2.5k,2.8k]
	Qwen3.5-9B	— (0 succ)	2.6k [2.4k,2.7k]	2.3k [2.1k,2.4k]
	Gemma-MoE-26B	— (0 succ)	3.4k [3.1k,3.6k]	2.5k [2.4k,2.7k]
	Qwen3.6-35B	— (0 succ)	3.2k [2.9k,3.4k]	3.0k [2.8k,3.2k]

Backup — RQ0.6 difficulty-binned success (object count)

task	bin	no-tools	+tool(steered)	Δ (pp)	n/arm
solve	≤6	40 [35,45]	95 [93,97]	+56	351
	6–14	16 [12,21]	71 [65,76]	+54	261
	>14	9 [6,13]	68 [63,73]	+60	288
validate_problem	≤5	35 [32,39]	95 [93,97]	+60	576
	5–13	35 [31,39]	91 [88,93]	+56	603
	>13	31 [27,35]	86 [83,89]	+55	540
validate_plan	≤6	46 [44,49]	85 [84,87]	+39	1755
	6–14	34 [31,36]	76 [74,79]	+43	1305
	>14	22 [20,25]	69 [66,71]	+46	1440
simulate	≤6	0 [0,1]	89 [85,92]	+89	351
	6–14	0 [0,1]	97 [95,99]	+97	261
	>14	0 [0,1]	93 [90,96]	+93	288